

User Technical Manual

Video Foreground Segmentation (ViFoSe)

MATLAB Application – App Designer

Authors

Filippo Piccinini: filippo.piccinini85@gmail.com

Luca Rinaldi: luca.rinaldi03@outlook.it

Michele Sarneri: michele.sarneri99@gmail.com

Abdelrahman Abdelaziz Mohamed: abdu.abdelaziz46@gmail.com

January 2026 Version

License

GNU General Public License v3.0

Table of Contents

1. Introduction
2. System Architecture
 - 2.1 Graphical User Interface Structure
 - 2.2 Internal Data Model
3. System Features
 - 3.1 Video Loading
 - 3.2 Definition of the Region of Interest (ROI)
 - 3.3 Manual Segmentation
 - 3.4 Selective Mask Reset
 - 3.5 Loading External Masks
 - 3.6 Project Saving
 - 3.7 Loading a Saved Project
 - 3.8 Alignment Functions
 - 3.9 Automatic Segmentation
4. Visualization and Update Algorithms
5. Implementation of the Output Background Color Selection Button

1. Introduction

ViFoSe (Video Foreground Segmentation) is an interactive application developed in the MATLAB R2024b environment or later, using the App Designer module. The main objective of the application is the manual and semi-automatic segmentation of video sequences through the use of binary masks, aimed at isolating regions of interest (ROIs) into foreground and background.

The application is designed to ensure a structured, reproducible, and modular workflow, integrating the following functionalities:

- video data pre-processing;
- interactive ROI selection;
- temporal and priority-based mask management;
- project state saving and restoration;
- interfacing with external modules for automatic segmentation and video stabilization.

2. System Architecture

2.1 Graphical User Interface Structure

The graphical user interface of the ViFoSe application is organized to support an intuitive and sequential use of the available functionalities. It consists of the following main elements:

Three graphical axes (UIAxes) arranged horizontally:

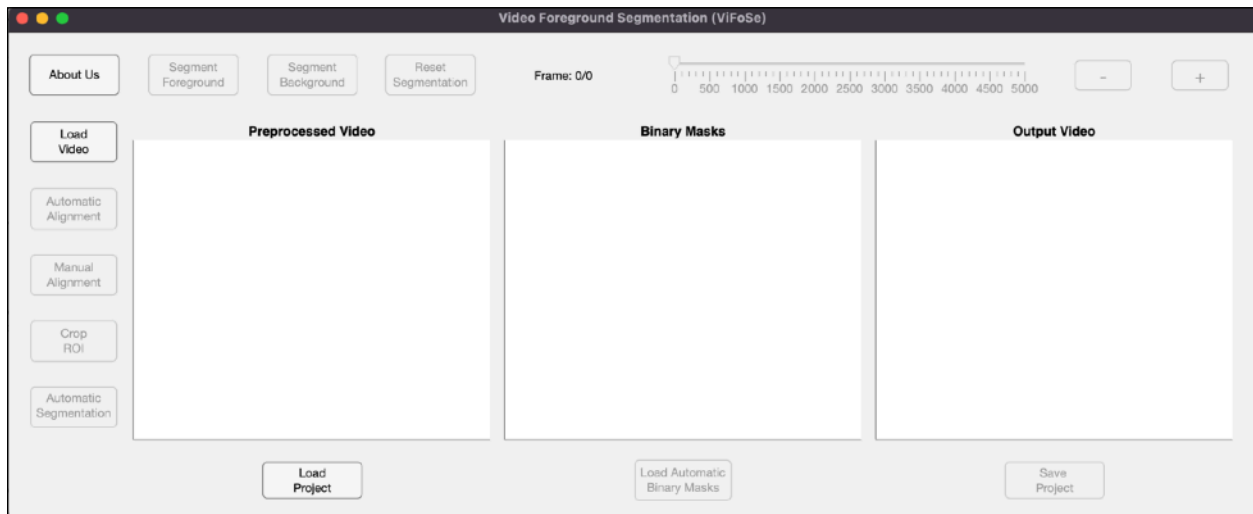
- UIAxes: display of the pre-processed video, with an optional ROI applied;
- UIAxes2: display of binary masks in grayscale;

- UIAxes3: display of the output video resulting from the segmentation.

A temporal navigation slider (Frame00Slider), accompanied by a dynamically updated label indicating the current frame and the total number of available frames.

A control panel, containing buttons organized by functional area:

- data loading and saving;
- temporal navigation;
- manual segmentation;
- advanced operations (alignment and automatic segmentation);
- informational section.



2.2 Internal Data Model

The application maintains in memory a set of state variables that constitute the internal data model. The main managed properties are:

- CurrentFrame: the current frame as an image matrix;

- ForegroundMaskTotal: cumulative set of foreground masks (3D array);
- BackgroundMaskTotal: cumulative set of background masks (3D array);
- ForegroundFrameIndices: indices of frames associated with foreground masks;
- BackgroundFrameIndices: indices of frames associated with background masks;
- ForegroundMaskTimestamps: creation timestamps of foreground masks;
- BackgroundMaskTimestamps: creation timestamps of background masks;
- CropRectROI: coordinates of the rectangular ROI [x, y, width, height];
- ExtractedFrames: cell array containing the extracted frames;
- VideoStructure: data structure of the original video;
- OutputFolder: path of the saving folder;
- FrameFilesList: ordered list of frame files;
- CurrentFrameIndex: index of the currently displayed frame.

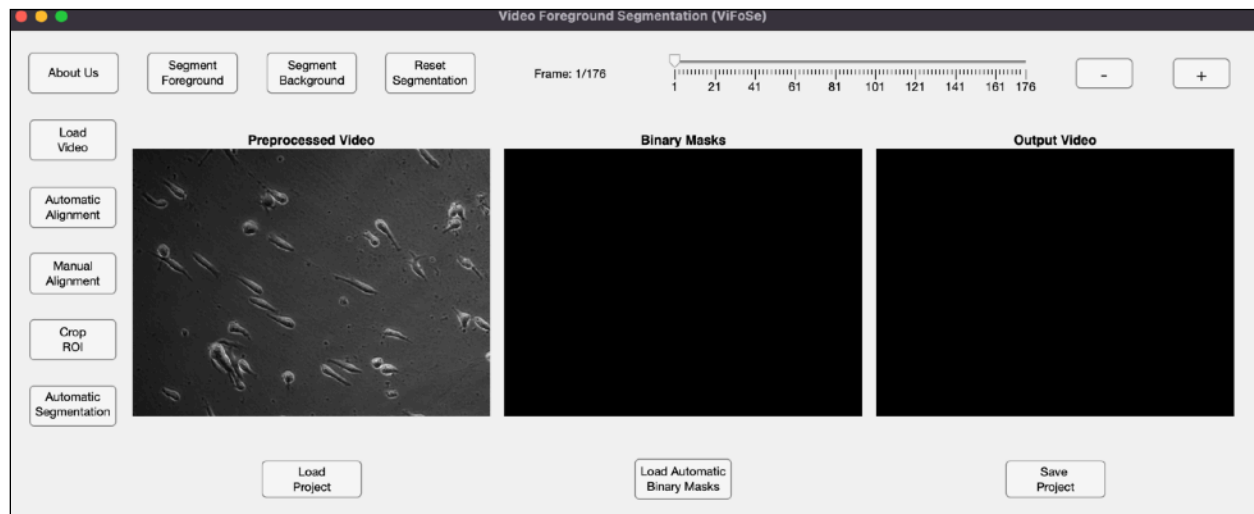
3. System Features

3.1 Video Loading

Video loading is performed via the LoadVideoButton. The associated function (LoadVideoButtonPushed) extracts frames from the selected video using the external function Grab_Video_Frames_par2.

The frames are saved as sequential PNG images within the subdirectory: Extracted_Frames/video_name/

This operation initializes the temporal slider, enables interactive components, and displays the first frame. Simultaneously, the binary masks are initialized and the first output frame is calculated.

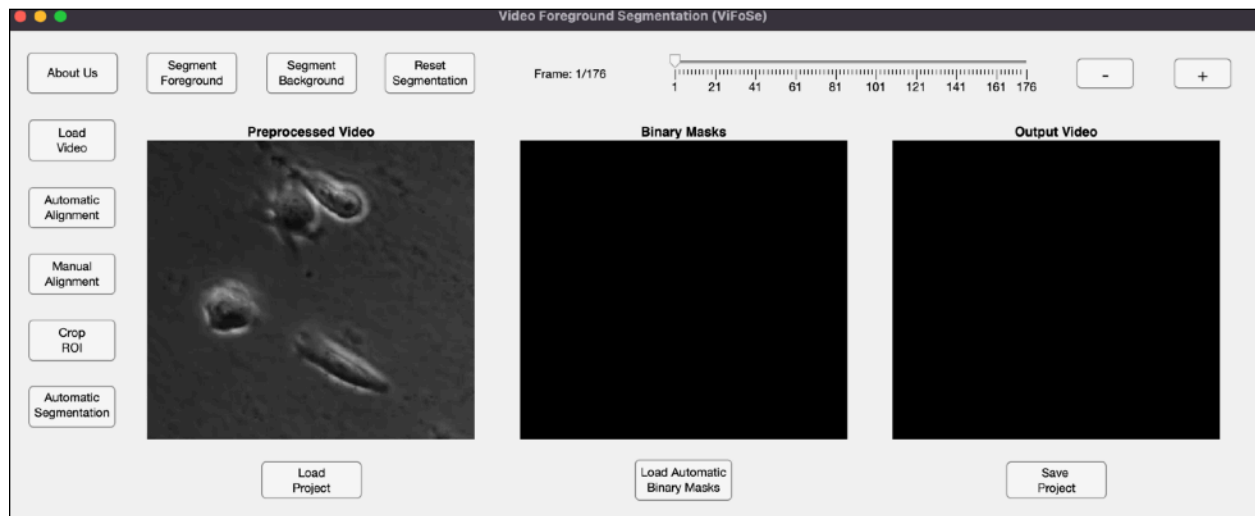


3.2 Definition of the Region of Interest (ROI)

The ROI is selected using the CropROIButton, which allows the interactive definition of a rectangular region on the current frame. The coordinates are stored in the CropRectROI variable.

All frames are cropped according to the selected ROI and saved in a dedicated folder. Existing masks are adjusted to the new dimensions through a consistent spatial transformation.

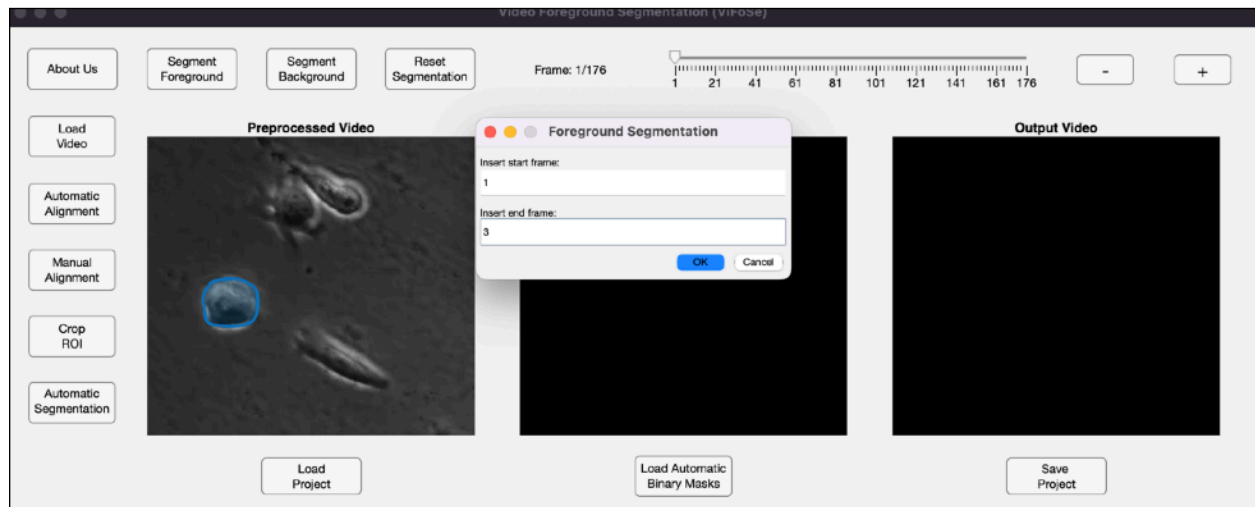


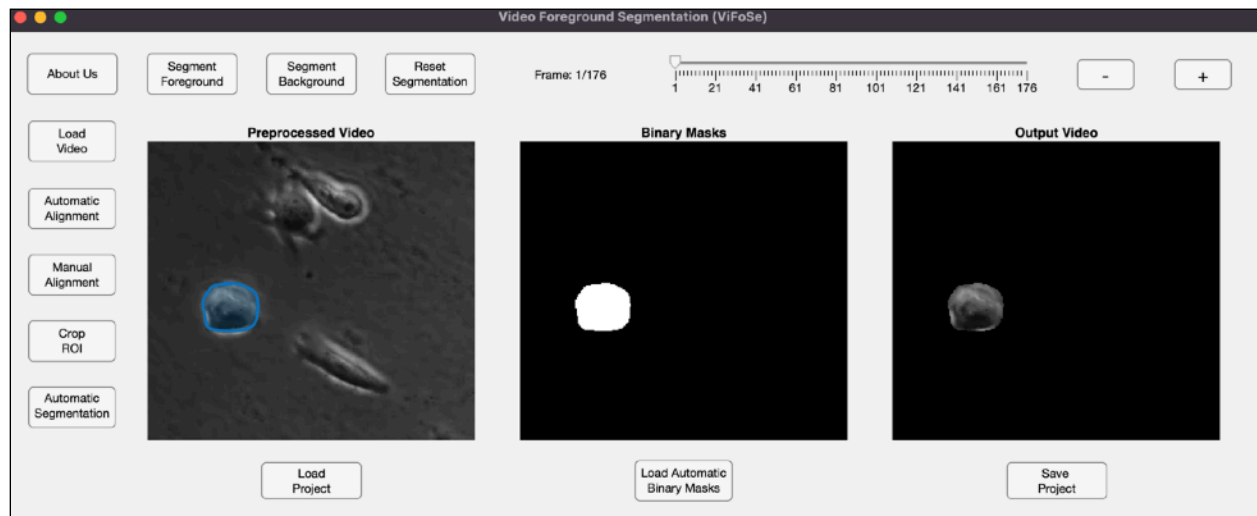


3.3 Manual Segmentation

Manual segmentation of the foreground and background is activated via the `SegmentForegroundButton` and `SegmentBackgroundButton`. The user draws a freehand region on the current frame, from which a binary mask is generated.

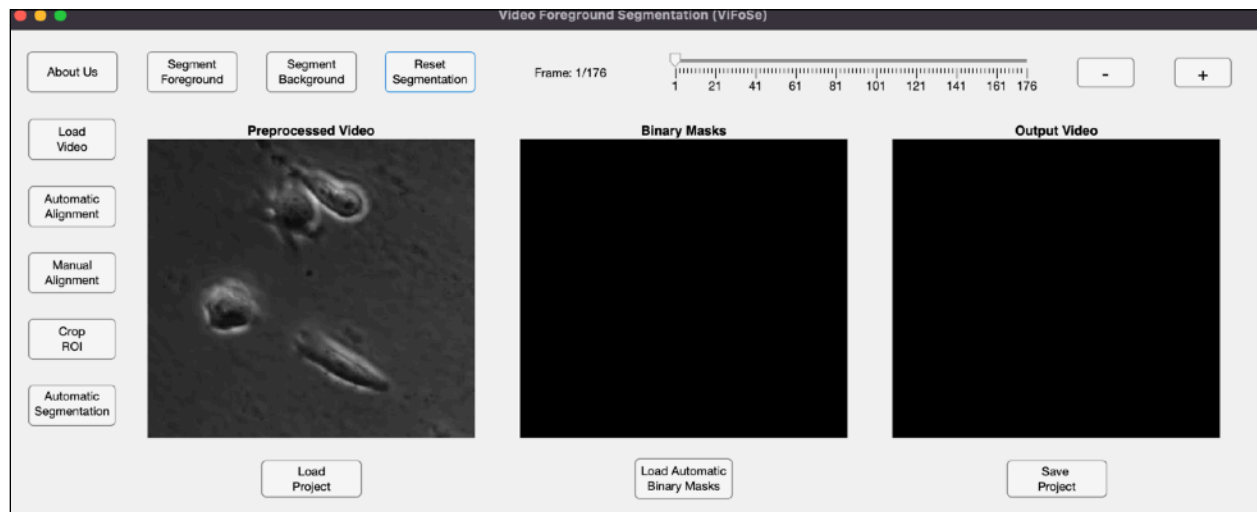
A dialog window allows specifying the temporal range for applying the mask, which is then associated with the corresponding frames and stored in the cumulative structures. Each mask is linked to a timestamp, used for managing temporal priorities.





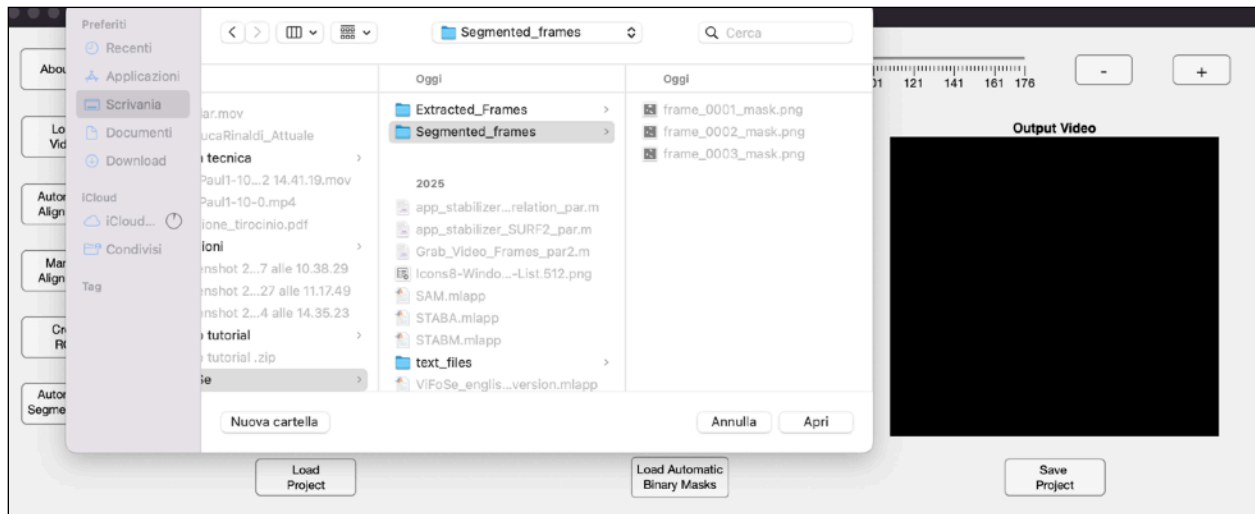
3.4 Selective Mask Reset

The selective reset allows the removal of masks within a user-defined frame range, while preserving masks outside that range.



3.5 Loading External Masks

It is possible to load sequences of external binary masks generated by automatic algorithms. The images are converted into binary masks and associated with a defined temporal range, after verifying format, order, and dimensional consistency.

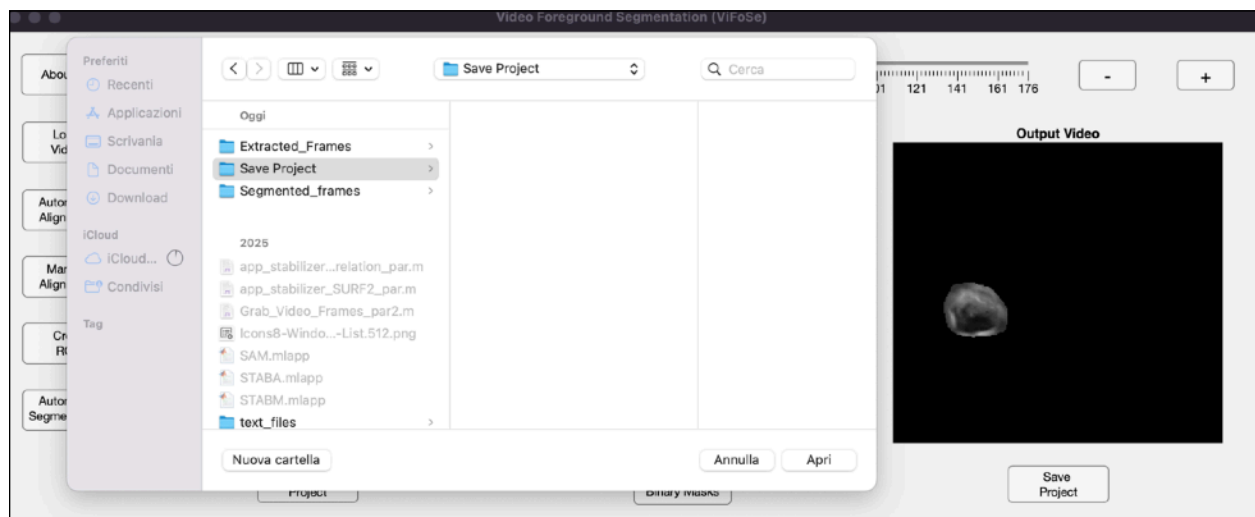


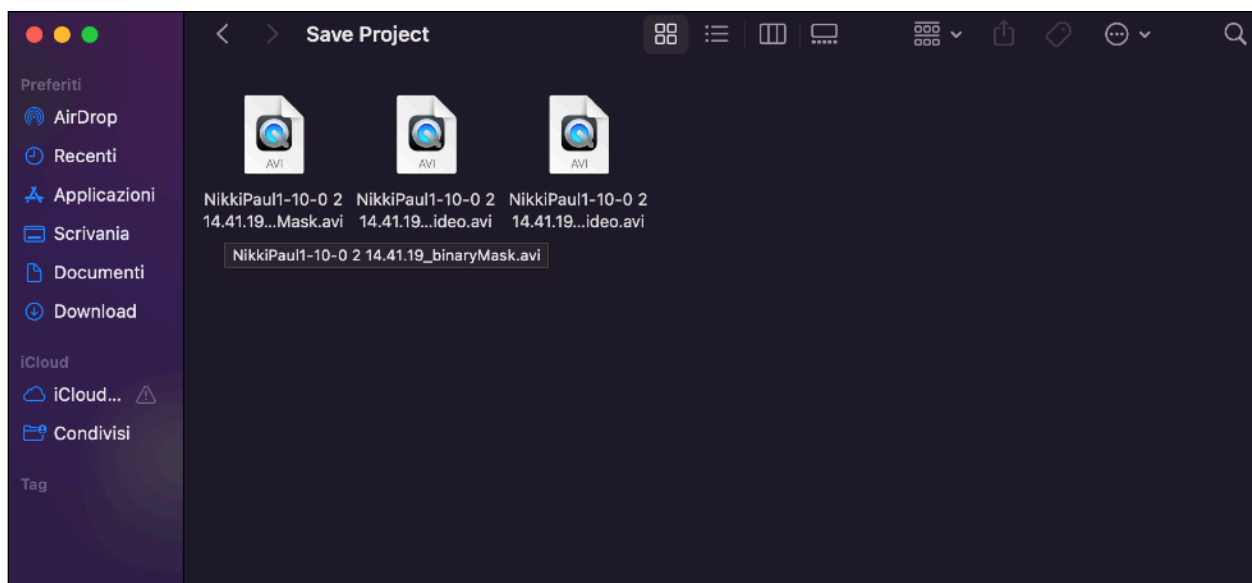
3.6 Project Saving

Project saving exports three videos in AVI format:

- pre-processed video;
- sequence of binary masks;
- segmented output video.

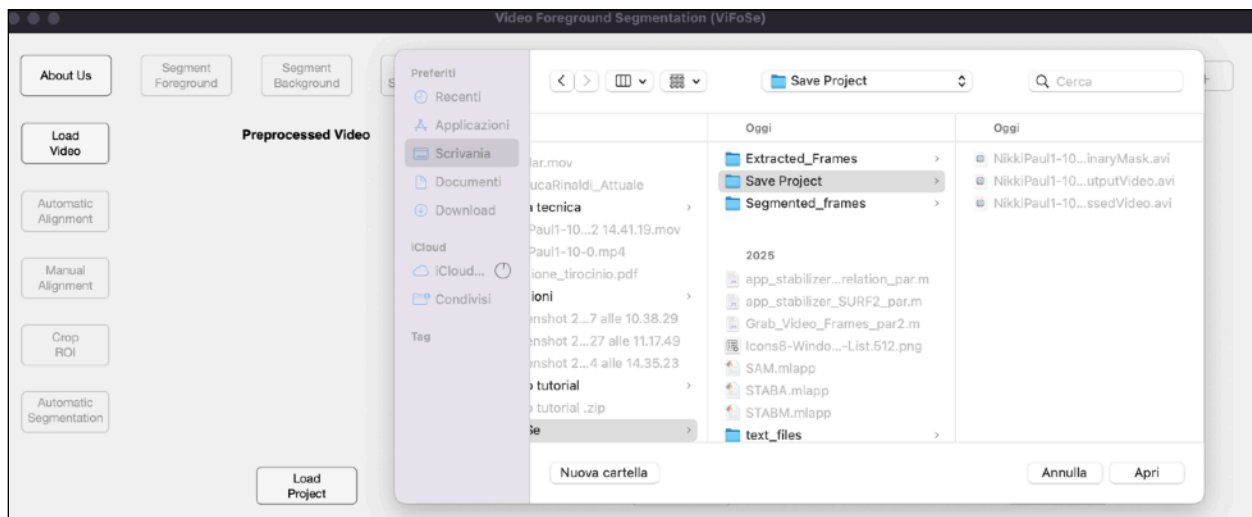
The frame rate is kept consistent with that of the original video.





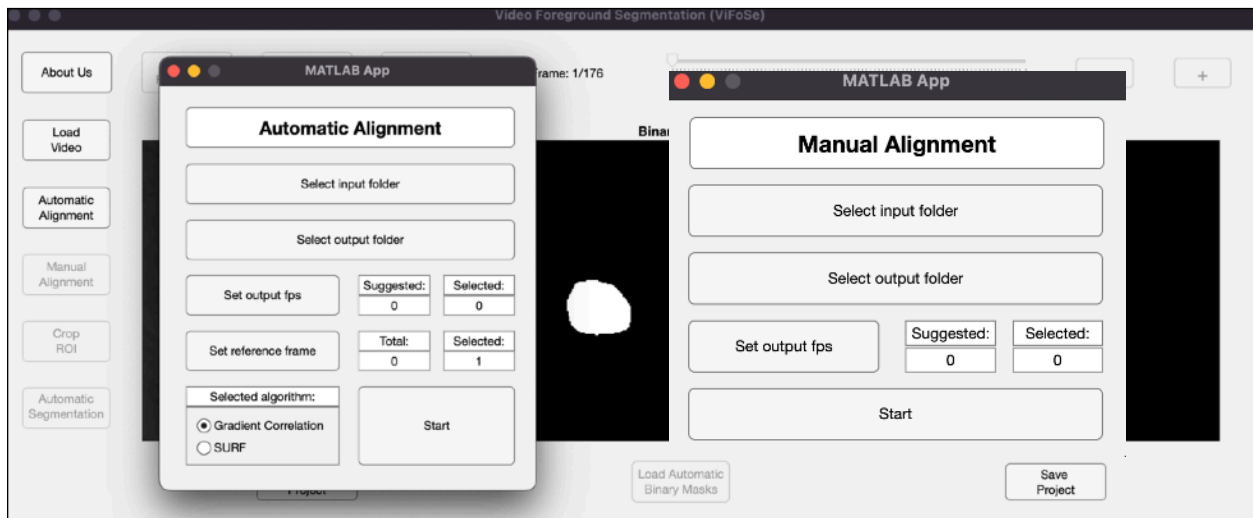
3.7 Loading a Saved Project

Loading a previously saved project allows the complete restoration of the application state, including masks, ROI, and temporal position.



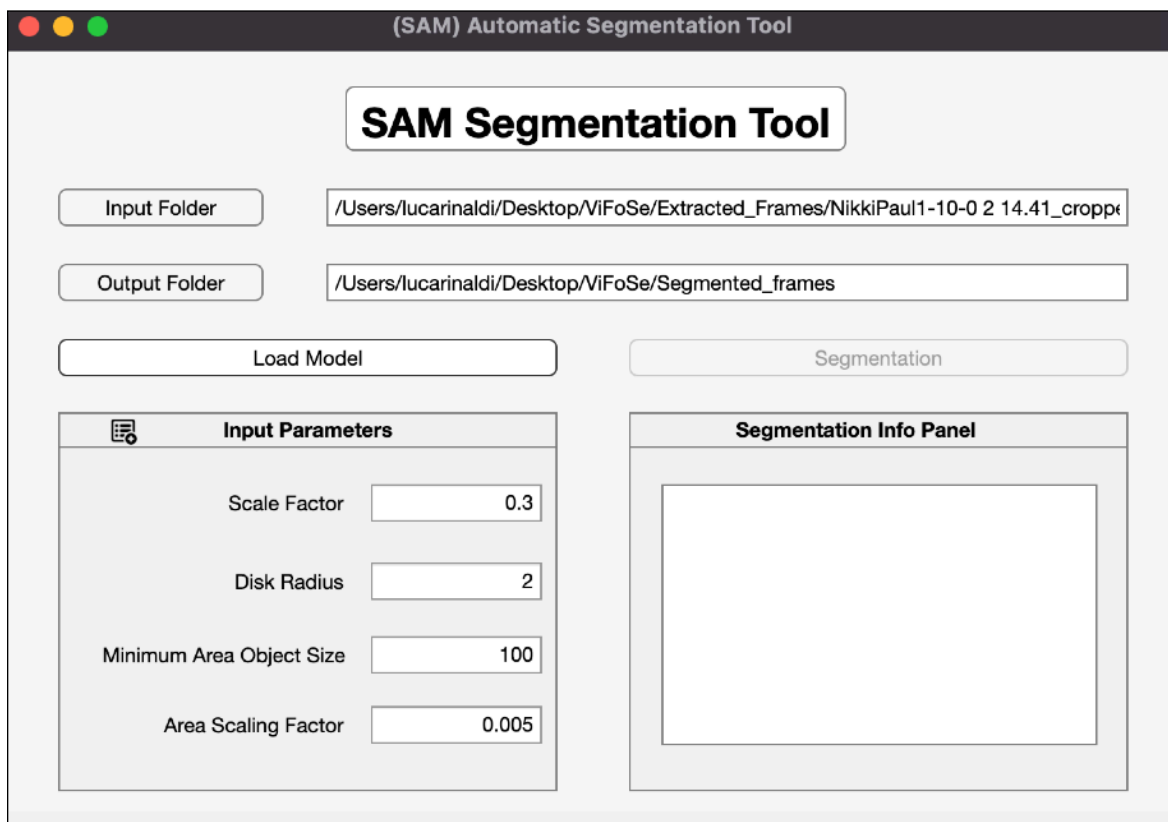
3.8 Alignment Functions

Automatic and manual alignment functions are available, which launch external applications dedicated to video stabilization.

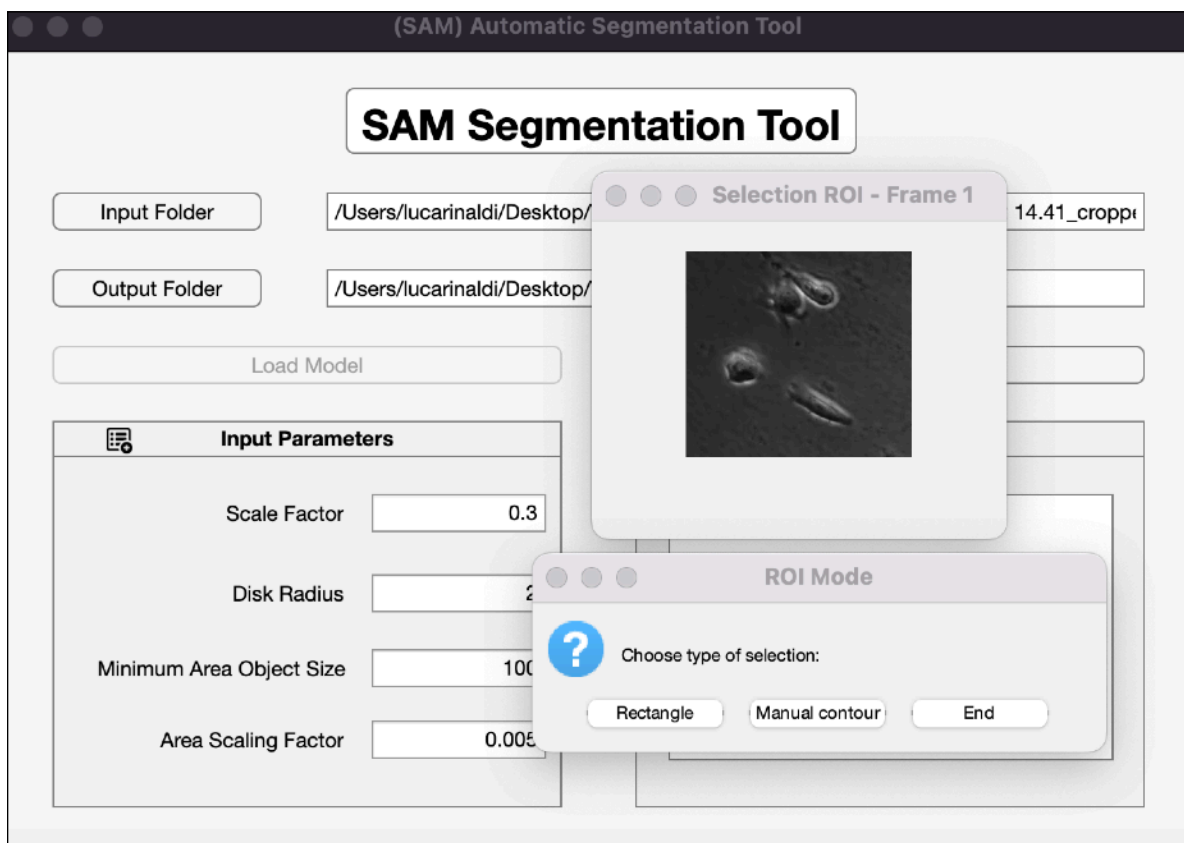


3.9 Automatic Segmentation

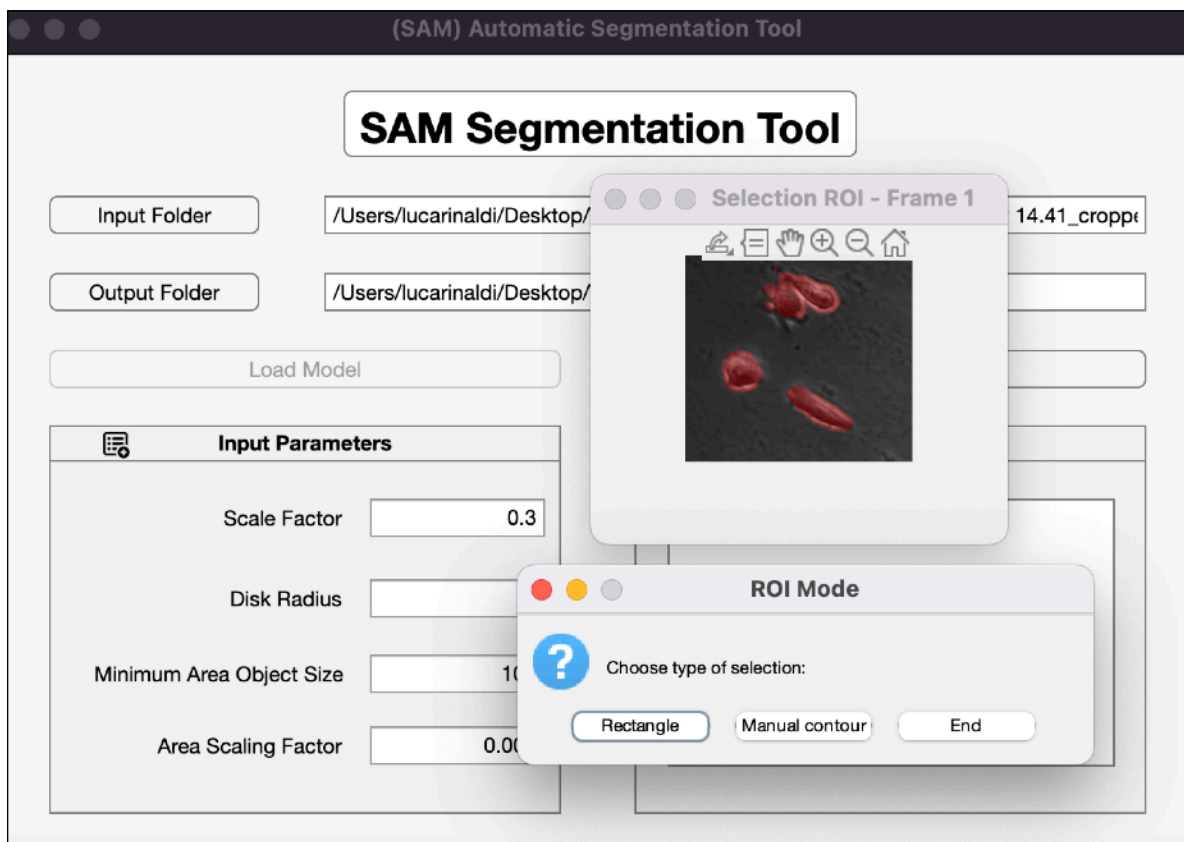
The application can interface with the Segment Anything Model (SAM) to perform automatic segmentation based on deep learning models.



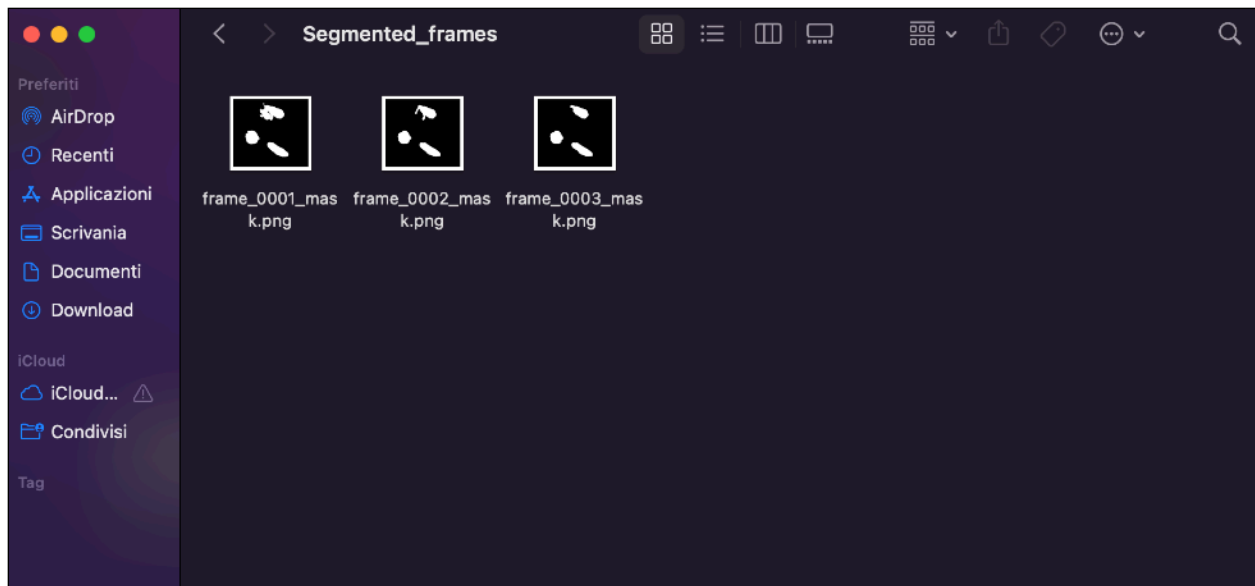
Input and Output Folder Selection



After Load Model and Segmentation: Automatic Segmentation Window Opens



Automatic Segmentation



Segmented Frames Saved in the Folder

4. System Requirements

ViFoSe requires MATLAB R2024b, with:

- Image Processing Toolbox.™;
- Deep Learning Toolbox.™;
- Mapping Toolbox.™;
- Medical Imaging Toolbox.™;
- Computer Vision Toolbox.™;
- Image Processing Toolbox Model for Segment Anything Model.™;
- Parallel Computing Toolbox.

All the materials described in this manual, including the ViFoSe application and associated resources, are fully available and can be accessed at the following link:

<https://github.com/UniBoDS4H/ViFoSe.git>

5. Implementation of the Output Background Color Selection Button

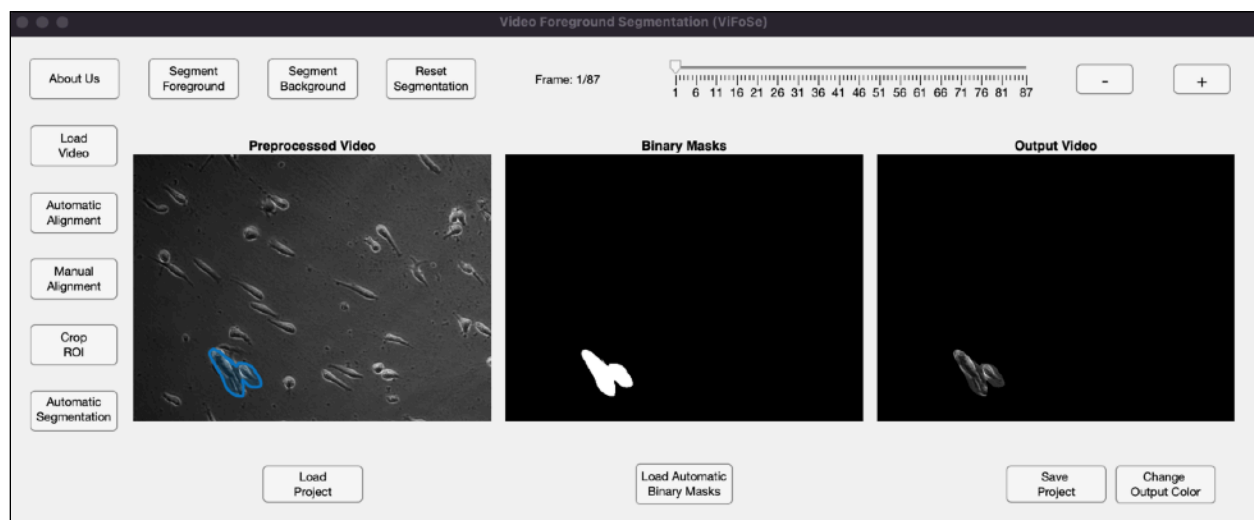
The implemented feature introduces a user interface control that allows dynamic modification of the background color of the output video display. When the dedicated button is activated, the application presents a modal selection dialog containing a predefined set of color options. The user can select a single color from the available list, after which the corresponding value is immediately applied to the output display.

The functionality is developed in MATLAB App Designer and relies on a pair of synchronized data structures that define both the user-visible color names and their associated RGB numeric values. Each displayed color label is directly mapped to a specific RGB triplet, ensuring a consistent relationship between the selected option and the rendered background color.

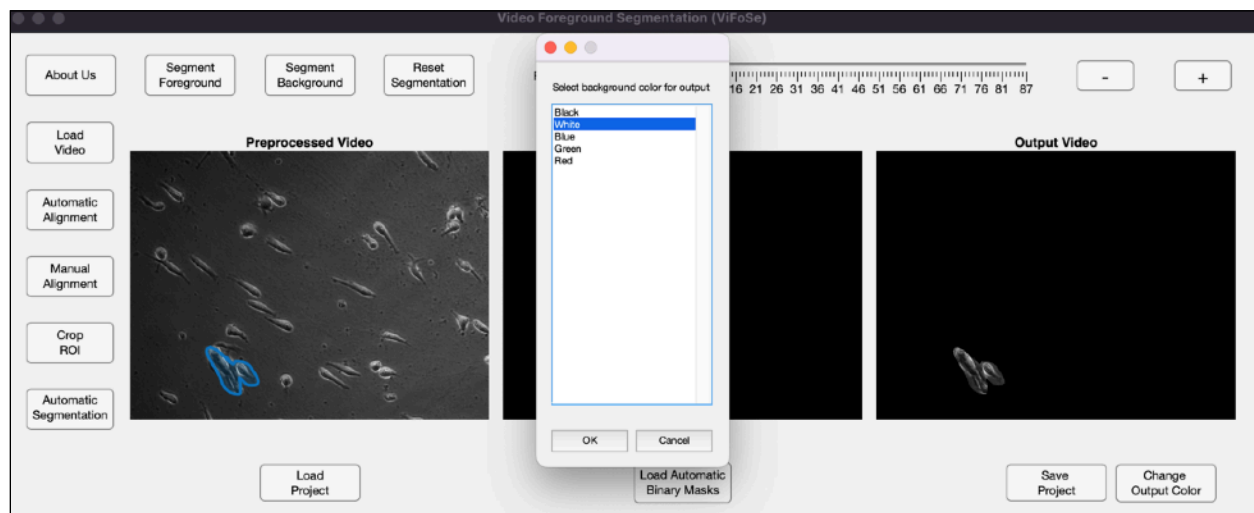
The selection interface is generated using a standard dialog window that enforces single-choice input. Once the user confirms the selection, the application verifies the validity of the action and retrieves the index of the chosen item. This index is then used to extract the corresponding RGB value, which is stored in the application state variable responsible for managing the output background color.

Following the update of the internal state, the application triggers a refresh of the output display component. This ensures that the newly selected background color is applied immediately to the video rendering area without requiring additional user interaction. The update mechanism is handled through a dedicated display refresh function that re-renders the UI axes content based on the updated configuration parameters.

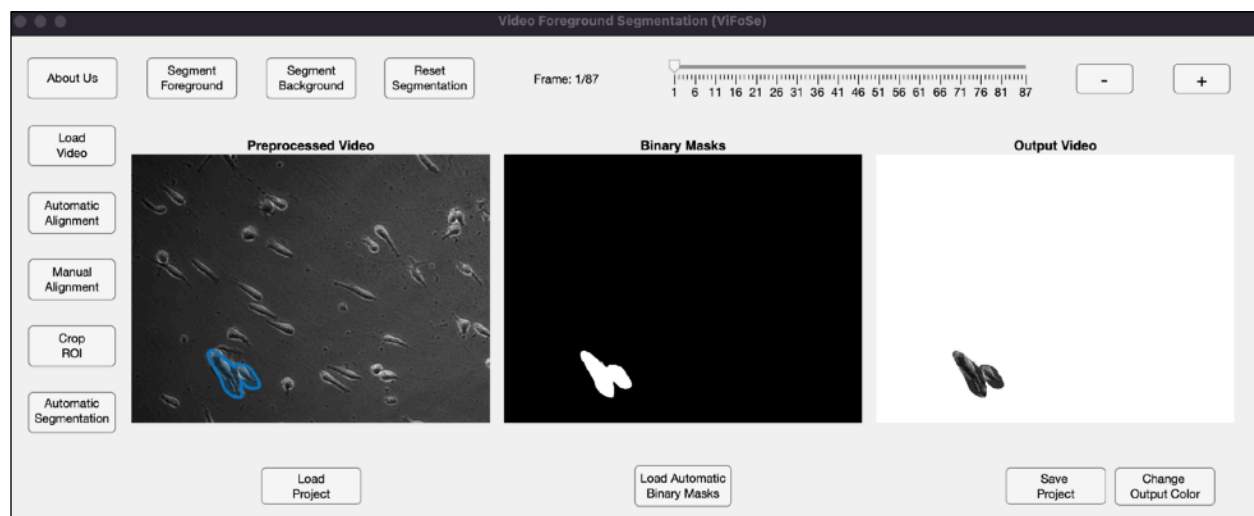
As a result, the system provides real-time customization of the video output background, improving the flexibility of the user interface and allowing immediate visual feedback following user selection.



Default black-background segmentation



Output color selection



Final segmentation with selected background color